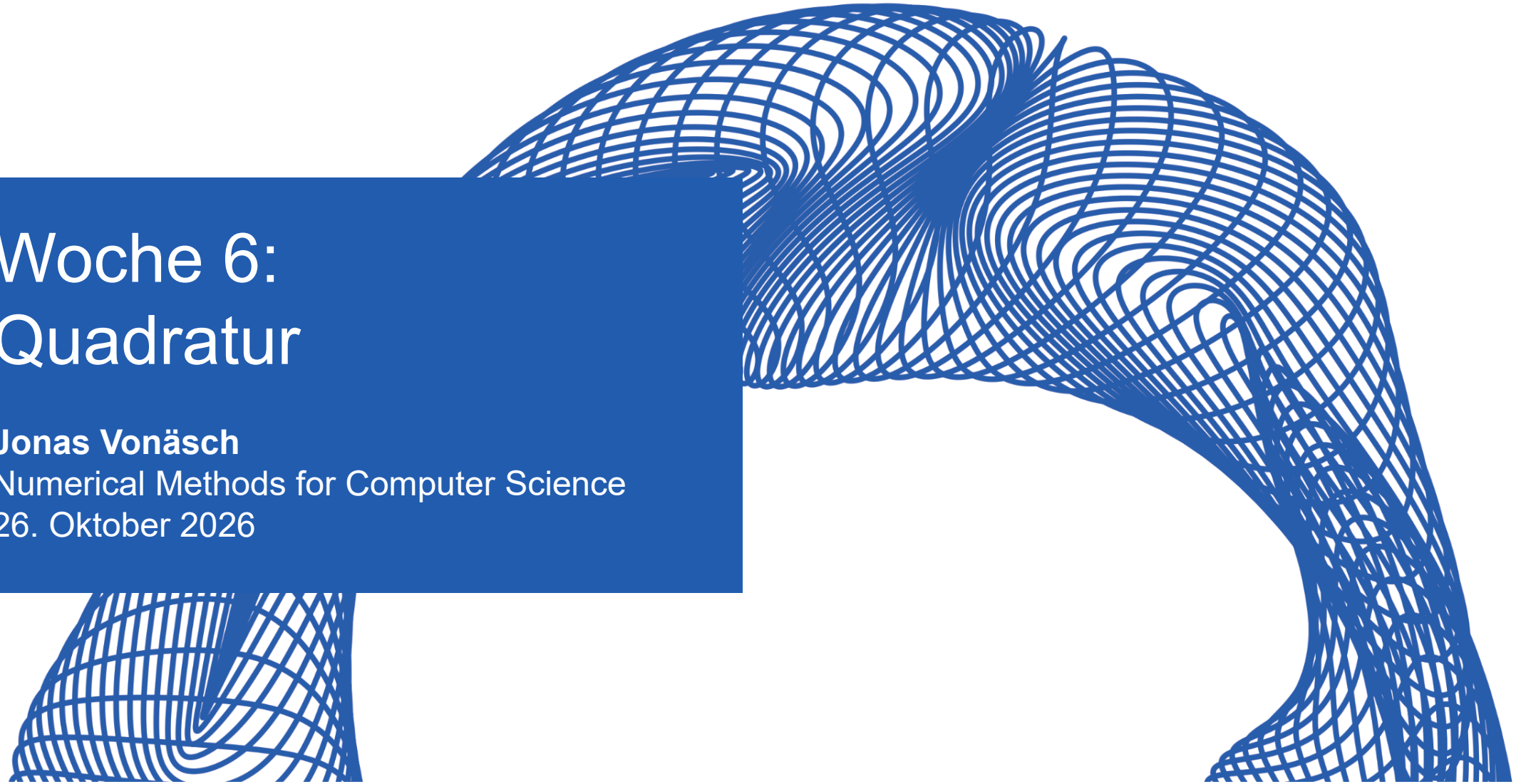
ETH zürich



Woche 6: Quadratur

Jonas Vonäsch

Numerical Methods for Computer Science

26. Oktober 2026

Agenda

1. Gauss-Quadratur
    1. Gauss-Legendre
    2. Gauss-Jakobi
    3. Gauss-Hermite
2. Clenshaw-Curtis-Quadratur
3. Recap Adaptive Quadratur?

4. Ausblick Programmieraufgaben

Woraus besteht jede Quadraturformel?

Stützstellen: $c_1, \ldots, c_n$

Gewichte: $w_1, \ldots, w_n$

Quadratur: $Q(f; a, b) = \sum_{k=1}^{n} f(c_k) w_k$

Fortsetzung

Bis jetzt:

$c_1, \ldots, c_N$ *äquidistant oder Adaptiv*

Heute:

$c_1, \ldots, c_N, w_1, \ldots, w_N$ *wählen, um Approximation zu verbessern*

Gauss-Quadratur

Gauss-Quadratur

Theorem 5.5.2. *(Maximale Ordnung einer Quadraturformel) Die Ordnung einer Quadraturformel mit s Knoten ist höchstens 2s.*

Ziel: Die Stützstellen neu wählen, sodass die Ordnung maximal wird!

Gauss-Quadratur: Herleitung

Teil des Inhalts der Folien zur Gauss-Herleitung von Leonard Schmidt

Wir approximieren das Integral:

$$\int_0^1 f\,(x)dx \approx \sum_{i=1}^{n} w_i\, f(c_i)$$

Wenn $w_1, \ldots, w_n, c_1, \ldots, c_n$ Monome mit Grad $\leq k$ exakt integrieren, integrieren wir

Auch Polynome mit Grad $\leq k$ exakt. Wir wollen:

$$\int_0^1 x^k\, dx = \sum_{i=1}^{n} w_i\, c_i^k$$

für möglichst grosses k

$2n$ unbekannte $(w_1, \ldots, w_n, c_1, \ldots, c_n) \Longrightarrow$ **Brauchen $2n$ Gleichungen!**

Gauss-Quadratur: Herleitung

Bedingung der Exaktheit für alle $k < 2n$:

$$\int_0^1 x^k \, dx = \sum_{i=1}^{n} w_i \, x_i^k$$

Damit haben wir schon unsere $2n$ Gleichungen. Eine Lösung des Systems wird Polynome vom Grad $\leq 2n - 1$ exakt approximieren.

Gauss-Quadratur: Herleitung Beispiel

Wir wollen $\int_0^1 f\,(x)dx$ mit $n = 2$ Knoten approximieren:

$$\int_0^1 x^k\,dx = \sum_{i=1}^{n} w_i\,x_i^k, \qquad k = 0,1,2,3$$

Unsere Integrale hier sind:

$$\int_0^1 x^0\,dx = 1, \quad \int_0^1 x^1\,dx = \frac{1}{2} \quad \int_0^1 x^2\,dx = \frac{1}{3} \quad \int_0^1 x^3\,dx = \frac{1}{4}$$

Wir erhalten das Gleichungssystem:

$$w_1 + w_2 = 1$$
$$w_1 x_1 + w_2 x_2 = \frac{1}{2}$$
$$w_1 x_1^2 + w_2 x_2^2 = \frac{1}{3}$$
$$w_1 x_1^3 + w_2 x_2^3 = \frac{1}{4}$$

Gauss-Quadratur: Orthogonale Polynome

Wir versuchen, das Gauss-Problem ohne nichtlineares Gleichungssystem zu lösen.

Definition (Orthogonale Polynome): Für Gewichtsfunktion $\omega(x)$ auf Intervall $[a, b]$ sei das gewichtete Skalarprodukt:

$$\langle f, g \rangle := \int_a^b f\,(x) g(x) \omega(x) dx$$

f und g sind orthogonal, wenn $\langle f, g \rangle = 0$

Gauss-Quadratur: Orthogonale Polynome

Theorem: Für jedes gewichtete Skalarprodukt existieren paarweise orthogonale Polynome:

$$p_0, p_1, p_2 \ldots, \qquad \deg(p_i) = i$$

Die Messstellen $c_1, \ldots, c_k$ sind genau die Nullstellen des Polynoms p_k. Diese Messstellen geben uns optimale Ordnung, wenn die Polynome alle orthogonal sind.

Gauss-Legendre-Quadratur

Gauss-Legendre-Quadratur: Definition

Wir haben Gewichtsfunktion $\omega(x) = 1$ auf $[-1,1]$

Die Legendre-Polynome berechnen wir wie folgt mit einer 3-Term-Rekursion:

- $p_{-1}(x) = 0$
- $p_0(x) = 1$
- $\beta_{k+1} = \frac{\langle x \cdot p_k, p_k \rangle}{\langle p_k, p_k \rangle}$
- $\gamma_k = \frac{\langle p_k, p_k \rangle}{\langle p_{k-1}, p_{k-1} \rangle}$
- $p_{k+1}(x) = (x - \beta_{k+1}) \cdot p_k(x) - \gamma_k p_{k-1}(x)$

Gauss-Legendre-Quadratur: Gewichte

Aus Knoten $c_1, \ldots, c_s$ erhalten wir Lagrange-Polynome $\ell_1, \ldots, \ell_s$.

$$\int_{-1}^{1} \ell_i\,(t)^2\,dt = \sum_{j=1}^{s} w_j \cdot \underbrace{\ell_i(c_j)^2}_{\substack{=0 \text{ für } j \neq i \\ =1 \text{ für } j=i}} = w_i \cdot 1 = w_i$$

Daraus folgt folgende Formel für die w_i:

$$w_i = \int_{-1}^{1} \ell_i\,(t)^2 dt$$

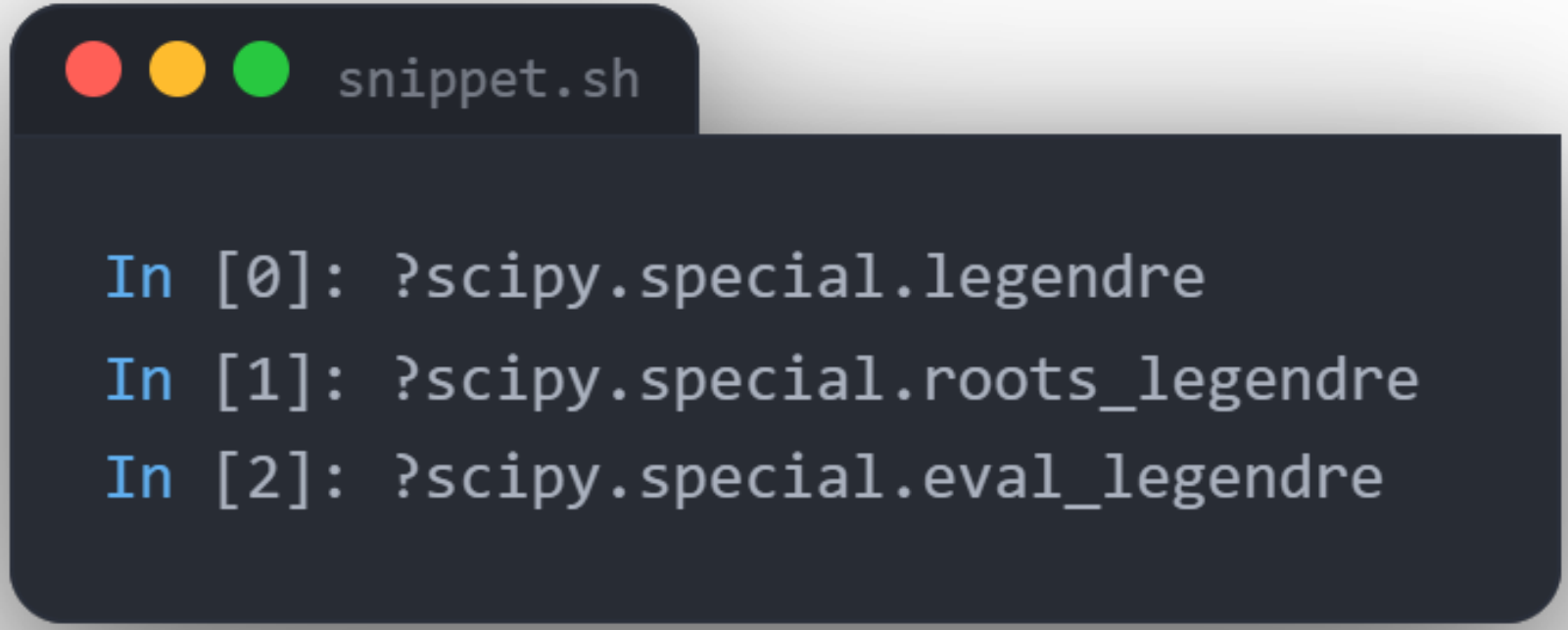
WTF??

Wieso verwenden wir die Nullstellen der Legendre-Polynome und Gewichte der Lagrange-Polynome?

- **Legendre-Nullstellen** garantieren Grad $2n - 1$

- **Lagrange-Basis** interpoliert die Polynome genau Gleich wie die Legendre-Basis – d.h. die Stützstellen sind hier auch optimal, aber einfacher für die Berechnung der Gewichte.

Gauss-Legendre-Quadratur in Praxis:



Gauss-Jakobi-Quadraturformeln

Jacobi-Polynome

Die **Legendre-Polynome** sind ein Spezialfall der **Jacobi-Polynome**. Die Jacobi-Polynome sind orthogonal bezüglich des Skalarprodukts:

$$\langle f, g \rangle = \int_{-1}^{1} f(x)g(x)(1-x)^{\alpha}(1+x)^{\beta}\, dx.$$

Durch geeignete Wahl von α und β erhalten wir verschiedene Quadraturtypen:

- $(\alpha, \beta) = (0, 0)$: Gauss-Legendre
- $(\alpha, \beta) = (1, 0)$: Gauss-Radau (rechter Randpunkt fest)
- $(\alpha, \beta) = (0, 1)$: Gauss-Radau (linker Randpunkt fest)
- $(\alpha, \beta) = (1, 1)$: Gauss-Lobatto (beide Randpunkte fest)

Sei $P_n^{(\alpha,\beta)}(x)$ das n-te Jacobi-Polynom bezüglich des obigen Skalarprodukts. Dann sind die optimalen Knoten für die entsprechende Gauss-Quadratur die Nullstellen von $P_n^{(\alpha,\beta)}(x)$.

Insbesondere: Gauss-Legendre nutzt die Nullstellen von $P_n^{(0,0)}(x)$.

Gauss-Hermite-Quadratur

Gauss-Hermite-Quadratur

Anstatt auf einem geschlossenen Intervall wollen wir nun auf $]-\infty, \infty[$ integrieren und suchen eine Formel für:

$$\int_{-\infty}^{\infty} f\,(x)\,\omega(x)\,dx \approx \sum_{i=1}^{n} w_i\,f(x_i)$$

Die Gewichtsfunktion sagt, dass uns das Zentrum mehr interessiert!

$$\omega(x) = e^{-x^2}$$

Hieraus konstruieren wir wieder unsere Polynome. Verwendet zum Programmieren:

`scipy.special.roots_hermite`

Clenshaw-Curtis-Quadratur

Clenshaw-Curtis-Quadratur: Idee

Wir können auch Quadratur wieder mit den Chebyshev-Polynomen in Laufzeit $O(n \log n)$ lösen.

Idee: Substituiere $x = \cos(\Theta)$

$$\int_{-1}^{1} f\,(x)dx = \int_{0}^{\pi} f\,(\cos(\Theta))sin(\Theta)d\Theta$$

- $c_1, \ldots, c_n$: Chebyshev-Abszissen
- Gewichte sind etwas komplizierter…

Clenshaw-Curtis-Quadratur

Wir definieren $F(\Theta) := f(\cos(\Theta))$ - F ist 2π-periodisch

⇒ **Cosinus-Transformation**

$$\int_0^{\pi} F(\Theta)\sin(\Theta)\,d\Theta = \int_0^{\pi} \sum_{k=0}^{\infty} a_k \cos(k\Theta)\sin(\Theta)\,d\Theta$$

$$= \sum_{k=0}^{\infty} a_k \int_0^{\pi} \cos(k\Theta)\sin(\Theta)\,d\Theta$$

$$= a_0 + \sum_{2 \le k \text{ gerade}} \frac{2a_k}{1-k^2}$$

Wobei a_k aus der Cosinus-Transformation stammen

Hier wieder wie bei Chebyshev: **Seht euch den Code im Skript an!**

Jupyter Notebook

Aufgaben

Quadratur auf Dreiecken

Quadraturregeln benutzen:

```
f = <irgendein Polynom>
rule = QRule(metname, p)
S = rule(<Eckpunkte TriVx>)
```

Dies gibt uns die Fläche des Polynoms p auf Dreieck `TriVx` angenähert mit Quadraturregel `<metname>`.

Nutzt **scipy.integrate.dblquad**, um die Scipy-Annäherung zu erhalten. Wenn die Differenz $<$ err_{tot} ist, nehmen wir eine exakte Berechnung an.

ETH zürich

Jonas Vonäsch
Teaching Assistant NumCS
jvonaesch@student.ethz.ch

jvonaesch.github.io